# StreamCSD: SSD-Autonomous Stream Management via In-Storage Content Learning

Wenjie Li[2*], Xiang Chen[1*],
Yelin Shan[2], Jiapin Wang[2], Yunxin Huang[2], Yafei Yang[2], Tao Lu[2✉], You Zhou[1], Fei Wu[1✉]
[1]Huazhong University of Science and Technology, [2]DapuStor Corporation
[*]Co-first authors, [✉]Corresponding authors: lvtao@dapustor.com, wufei@mail.hust.edu.cn

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11133202

***Abstract*—Write amplification (WA) from migrating valid pages during garbage collection (GC) degrades SSD performance and lifespan. Although stream management based on high-level software semantics reduces WA, existing solutions require host modifications, hindering their adoption. We introduce StreamCSD, an SSD-autonomous stream management approach using in-storage content learning, eliminating host-side changes. Leveraging compression ratios from embedded compressors in computational storage drives (CSDs), StreamCSD employs a streaming K-means algorithm to cost-efficiently cluster data into streams. Evaluations show that StreamCSD reduces WA from 1.7 to 1.06 under multimodal generative AI workloads, matching state-of-the-art methods with minimal impact on bandwidth. StreamCSD operates without host modifications, promoting broader adoption of multi-stream SSDs.**

## I. Introduction

SSDs in data centers typically serve multiple workloads, such as databases, virtual machines, and file storage. These workloads produce writes, updates, and deletions, each with distinct patterns and timing. Consequently, the lifetime of the data is inconsistent. A standard block-interface SSD cannot distinguish between pages from different write streams due to a lack of semantic information. As a result, pages within a flash erase block have varying lifetimes, leading to the migration of many valid pages during garbage collection (GC) [21], [22] and causing write amplification (WA), which degrades SSD performance and requires over-provisioned flash resources for mitigation, which are major sources of the *block interface tax* [9].

Effectively mitigating WA involves storing pages with similar lifetimes in the same physical stream or block group within SSDs. However, accurately predicting page lifetime is a challenge due to workload variability and dynamics [19], [23]. In practice, instead of relying directly on lifetime, existing approaches utilize programmer hints through new IO interfaces [9], [10] and stream IDs based on data types [17] and program contexts [19], which are semantically or empirically linked to data lifetimes [28], to manage streams.

Open-Channel [10] and Zoned Namespace (ZNS) [9] revamp the block interface, empowering the host with direct control over the placement of data in flash blocks. However, they are incompatible with systems and applications designed for block interfaces. In contrast, multi-stream SSDs [17], [19], [25] and Flexible Data Placement (FDP) [27] enhance the existing block interface by allowing the host to use software semantics to establish distinct write streams. This enables SSDs to judiciously place data with the same stream ID across separate flash blocks. However, these solutions require extensions to the host-SSD communication protocol. This necessitates modifications to NVMe drivers or applications to include stream or placement identifiers, presenting a barrier to adopting these stream management schemes.

The interface changes or host dependencies result in significant limitations to existing schemes. As a supplier of data center SSDs, more than 95% of our SSD shipments consist of traditional block-interface SSDs. Given the still prevailing use of block-interface SSDs, we raise a critical and fundamental question: Can we retain block interfaces while eliminating host dependencies to devise a fully SSD-autonomous, cost-efficient stream management mechanism for reducing GC-induced WA?

We introduce *Learned Streaming*, an SSD-autonomous stream management approach that leverages content as a novel semantic dimension to eliminate host dependency. However, learned streaming requires data feature extraction, which presents challenges in terms of *universality* and *efficiency*. For instance, traditional content clustering algorithms, such as TF-IDF [8], are limited to text formats (e.g., txt, JSON) and suffer from slow execution speeds, making them unsuitable for latency-sensitive SSD stream management.

We propose *StreamCSD* to address the above challenges. By using Shannon entropy (ENT) or compression ratio (CR) as features, *StreamCSD* can differentiate a wide range of streams with varying lifetimes. By leveraging the computational storage drive (CSD) [32] architecture and piggybacking page feature extraction on SSD hardware compressors (emerging SSDs tend to integrate hardware compressors to reduce storage cost and prolong lifetime) [11], [29], [31], *StreamCSD* introduces negligible overhead. Furthermore, for SSDs without built-in compressors, the overhead can be mitigated by implementing a Shannon entropy accelerator. Following feature extraction, *StreamCSD* employs a lightweight *streaming K-means* algorithm for feature clustering and stream assignment. *Learned Streaming* based on the page compression ratio also faces the challenge from two aspects. First, pages sharing the same compression ratio might have different lifetimes. Second, pages with different compression ratios may have similar lifetimes. These limitations compromise the goal of separating data with different lifetimes. We further propose GC-guided page-to-stream remapping (§IV-C) and secondary lifetime-assisted stream mapping (§IV-D) to complement *Learned Streaming*.

We implement *StreamCSD* with the *Learned Streaming* mechanism on both the FEMU emulator [20] and a PCIe 5.0 ×4-lane SSD with a built-in hardware compressor. Experimental results show that *StreamCSD* reduces WA by 63.4% and improves IO performance by 74% compared to baseline, matching or surpassing state-of-the-art methods including *PCStream* [19] and MiDAS [23]. Under multimodal generative AI workloads, StreamCSD lowers the WA from 1.7 to 1.06 while having a negligible effect on write bandwidth.

## II. Background and Related Work

The block interface tax [9], [10] arises from the mismatch between the block interface and flash memory's characteristics. To bridge this gap, numerous mechanisms have been proposed. Open-channel [10], [24] and ZNS [9], [15] interfaces expose different levels of flash memory management to the host. The multi-stream [17] and FDP [27], [34] allow hosts to deliver hints of data placement to SSDs.

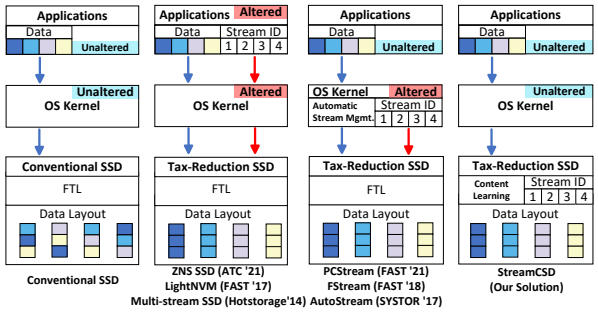


Fig. 1: SSD block interface tax reduction mechanisms. StreamCSD targets a host-transparent multi-stream SSD with a block interface.

Previous works have exploited various software hints through the multi-stream interface, such as multi-level semantics of LSM-tree [17], data types in file systems [25], and program contexts at the application layer [19]. LBA-based clustering methods classify page hotness by analyzing historical LBA write patterns, including recency, frequency, sequentiality and so on.

Figure 1 illustrates the host dependencies of existing mechanisms. Open-channel [10] and ZNS [9] interfaces require significant modifications to the application and OS kernel. The open-channel interface exposes raw NAND details, forcing the host to manage tasks like data mapping, garbage collection, and wear leveling. ZNS requires the host to manage sequential writes, zone resets, and garbage collection while the SSD firmware handles wear leveling. Similarly, Multi-stream [17] and FDP [27] require the application and OS kernel to assign stream or placement IDs, demanding application-specific knowledge to abstract different write streams. Although techniques like PCStream [19], FStream [25], and AutoStream [39] can autonomously extract application streams, they still necessitate OS kernel modifications. LBA-based clustering can be integrated directly into SSDs, independent of the host. However, these methods often require extensive statistics, such as LBA update intervals and block GC survival counts, or even develop complex models [23], [39] for clustering. These methods consume non-trivial amounts of valuable CPU and memory resources in SSDs, and rely on LBA access locality, making them unsuitable for frequently changing IO patterns and log-structured systems (e.g., RocksDB and F2FS) [19].

## III. Motivation: Content-based Stream Management

### A. Opportunities and Challenges

Content-based clustering, a robust unsupervised learning technique, is extensively employed in recommendation systems and big data analysis to group objects like documents [33], images [12], and videos [7] based on similarity or relevance. Techniques such as the *K-means* algorithm exploit term frequency-inverse document frequency (TF-IDF) vectors for clustering documents [8]. Inspired by these methods, we investigate the application of content learning within SSDs to enhance data placement and reduce write amplification.

However, implementing content-based clustering for intra-SSD stream management faces significant challenges. Current algorithms, designed for specific types of data such as documents and images, cannot be universally applied across the varied data handled by SSDs. Furthermore, the computational overhead of these clustering methods, particularly the high-dimensional processing and vectorization in TF-IDF, imposes a substantial CPU burden, demonstrated by a latency of 0.24 seconds for clustering 64 pages (or 4ms per page). This can severely impact SSD performance. Therefore, a universal and efficient content-based clustering algorithm is essential to effectively differentiate data streams without overwhelming the SSD resources.

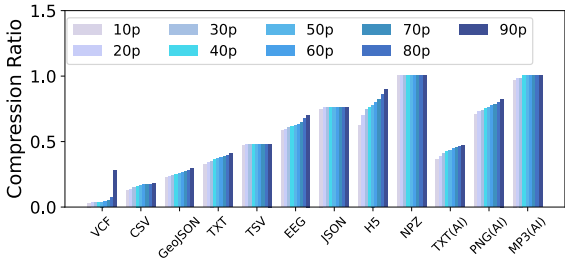


Fig. 2: Percentile distribution of 4KB page compression ratio in real datasets. '10p, 20p' refers to the 10th, 20th percentile CR and etc. These datasets are also employed to fill the microbenchmark as described in Table I and will be published.

### B. Compression Ratio: A Good Multi-Stream Indicator?

**Compression ratio feature.** Typically, different types of data content are often managed independently, resulting in distinct write streams. For instance, file systems and databases typically separate user data, system metadata, and recovery logs into different streams. Similarly, multimodal generative AI applications produce a variety of content types—such as text, images, audio, and video—each stored in separate streams based on its unique structural characteristics. These structural differences, or content patterns, play a critical role in determining data compressibility. Compression algorithms exploit these patterns to minimize data size by replacing repeated sequences with pointers (e.g., LZ [41]), using shorter codes for frequent symbols and longer ones for rare symbols (e.g., entropy coders [16]). Therefore, the compression ratio is potentially a valuable indicator of the internal pattern and structure of data content.

**Compression ratio and stream ID.** To demonstrate whether compression ratio or entropy is a good indicator for distinguishing streams, we analyze 9 domain-various datasets from a public repository [1] and 3 AI-generated datasets [3]. Each dataset corresponds to a write stream and is split into 4KB pages, which are compressed to quantify percentile distributions of compression ratios. As depicted in Figure 2, the CRs of pages within each stream converge closely, with a mean deviation of only 0.02, which indicates *compressibility locality*. A similar observation is made in a recent work [36]. We also observe that the CRs or entropy values of pages diverge across different write streams. As Figure 3 illustrates, 9 of the 12 datasets exhibit significant distinction in either compression ratio or entropy value, highlighting a strong correlation between compression ratio and entropy. From the above results, we can see both the compression ratio and entropy are universal and representative features effective enough for content-based data page clustering.

**Compression ratio and data lifetime.** As shown in Figure 4, entropy strongly correlates with data lifetime in many important scenarios, and this also applies to the compression ratio. RocksDB's Write-Ahead Logs (WALs) exhibit lower entropy values or compression ratios and shorter lifetimes compared to Sorted String Tables (SSTables) [19]. WALs avoid compression to minimize latency [5], while SSTables use Snappy compression to reduce storage usage. Similar patterns are seen in Cassandra's flushing and compaction SSTables [2]. The redo and DB files in MySQL are difficult to distinguish based on entropy. However, for the overall data, those with lower entropy values tend to have shorter lifetimes, while data with higher entropy values tend to have longer lifetimes. In GCC compilation, intermediate files like preprocessed and assembly files have different entropy and compression ratios compared to long-lived object files. The correlation between file extensions and compression ratios has also been proven effective for stream separation [17], [25].

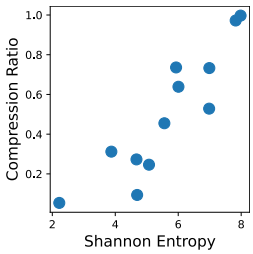


Fig. 3: Compression ratios and Shannon entropy.

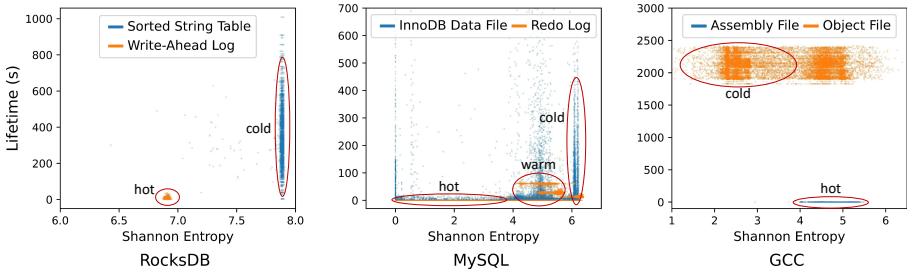


Fig. 4: Correlation between Shannon entropy and data lifetime in representative databases and Linux kernel source code compiling applications.

## IV. Design of StreamCSD

### A. Overview

To alleviate the block interface tax, we propose a computational SSD, called *StreamCSD*, which employs a *learned streaming* mechanism. It bears the following design goals:

- **G1: Host-transparent in-storage multi-streaming.** StreamCSD presents a standard block device and infers stream ID based on page content for automatic multi-streaming to reduce GC overhead. No application-specific expertise and host software modifications are required.
- **G2: Real-time multi-streaming.** StreamCSD can perform real-time identification of write streams among incoming data pages on the write IO path, eliminating the need to track and analyze historical access patterns of LBAs.
- **G3: Comparable efficacy to existing approaches.** StreamCSD should be comparable to or even outperform host-guided multi-stream approaches and LBA-based clustering approaches in representative application scenarios.
- **G4: Free from side effects.** StreamCSD should be lightweight with insignificant overheads. Even when the workload is unfriendly for StreamCSD, the performance and WA should not degrade.

Figure 5 presents the design overview of StreamCSD, which integrates a content learning accelerator for real-time page feature extraction. The *Shannon entropy* or *compression ratio* can be used as the learned feature, which is consumed by the streaming K-means algorithm to separate incoming data pages into different learned streams (§IV-B). The basic workflow of StreamCSD is as follows. ❶ The host NVMe write request is transmitted into the SSD via the PCIe interface. ❷ The NVMe controller inside the SSD parses the request. ❸ Each data page is transferred into the data cache of the content learning accelerator via a high-speed bus for content learning and feature extraction. These feature values are processed through clustering to obtain the corresponding stream ID (LearnedID). ❹ The data pages along with their stream IDs are written into the SSD write buffer. ❺ The FTL flushes data pages with different stream IDs to the corresponding physical blocks or reclaim units.

However, pages originating from different applications, with varying lifetimes but similar compression ratios, may be clustered into the same stream. StreamCSD employs a *GC-guided stream remapping* mechanism (§IV-C) to overcome this issue. Moreover, multi-stream SSDs support a limited number of physical streams, typically 4 to 16 [9], [19], [39], but high-concurrent application scenarios can generate more learned streams than available physical streams. To resolve this mismatch, we propose a *secondary lifetime-assisted stream mapping* mechanism that efficiently merges learned streams to fit the available physical streams (§IV-D).

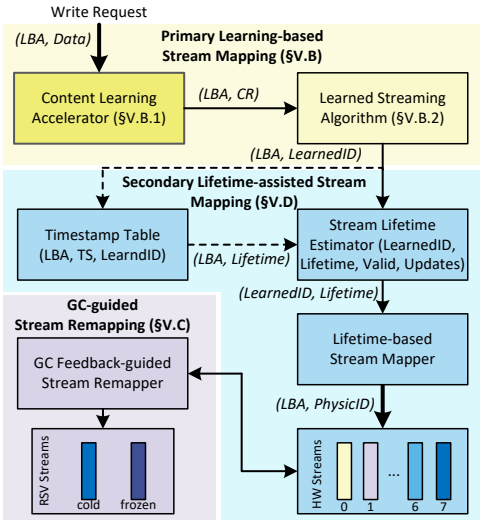


Fig. 5: The design overview of StreamCSD. Note, the secondary lifetime-assisted stream mapping mechanism is conditional. When it is not activated, learned IDs directly map to physical stream IDs.

### B. Primary Page Content Learning-based Stream Mapping

Compression ratio and Shannon entropy, as universal data-characterizing features, yield similar classification accuracy. However, relying on the SSD controller's ARM cores for compression or Shannon entropy calculation can create performance bottlenecks in the write IO path. To address this, we propose using hardware acceleration and a learned streaming algorithm within the SSD as an effective solution.

*1) Content Learning Accelerator:* Accurately determining the stream for each page is critical for host-transparent multi-stream SSD. Efficiency is crucial in designing the feature extraction algorithm to avoid a significant impact on SSD write performance.

**Piggyback feature extraction on compressors.** Some SSDs have integrated hardware compressors [29], [31] for storage savings. In these instances, *StreamCSD* can utilize the SSD's internal compressor to extract page features, thereby eliminating the need for an additional content-learning accelerator. Consequently, *StreamCSD* leverages the page compression ratio as a primary metric in identifying page stream ID, without incurring extra computational costs.

**Shannon entropy accelerator.** In SSDs without built-in compressors, implementing a Shannon entropy accelerator for feature extraction is recommended, as it is significantly simpler than a compression engine [6], [18]. The hardware engine for entropy computation efficiently analyzes symbol (byte) frequency in input data. It consists of a data buffer for input storage, a counter array to monitor symbol frequencies, a probability calculator, a logarithm unit

utilizing an approximation technique, and an accumulator to handle the multiplication of probabilities and logarithms. In our simulation on a *Vivado XCVU19P FSVA3824* FPGA, the entropy module is remarkably compact, taking up less than 1% of the area relative to embedded processor cores. The computation for a 4KB page is accomplished in 1024 cycles, corresponding to 1$\mu$s at a 1GHz core frequency. Notably, a 6.25% sampling of 256 bytes achieves 98% entropy accuracy with a calculation time of only 0.05$\mu$s, ensuring minimal impact on SSD performance.

*2) Learned Streaming Algorithm:* **Streaming K-means clustering.** For real-time efficiency in SSD multi-stream identification, we employ a streaming K-means algorithm [30] to cluster learned streams. Unlike traditional K-means, which processes data in batches, streaming K-means continuously updates clusters and assigns stream IDs as new data arrives. The algorithm begins by specifying the number of clusters ($K$) and initializing centroids to maximize the minimum distance between them [14]. As new pages arrive, the algorithm calculates distances based on accelerator-generated features, assigns stream IDs to each page, and updates centroids when the predefined batch size is reached.

**Wait-free clustering for single-page.** Page features and stream IDs are cached until a predefined threshold triggers a batch update to the K-means centroids. Notably, our customized K-means algorithm assigns stream IDs on a per-page basis, eliminating clustering wait time. This separation of model updates from stream ID assignments ensures immediate stream ID retrieval, which is crucial for continuous data influx. As a result, tasks such as FTL address translation and backend data flushing can proceed without delay. Combining hardware-accelerated features of compression ratio or entropy, this approach ensures immediate page flushing to NAND flash with readily available stream IDs.

### C. GC-guided Stream Remapping

Learning-based stream mapping is effective for streams with consistent lifetimes, such as WAL or SSTables. However, non-uniform workload distributions (e.g., *Zipfian*, *latest*) [13], and limitations of unsupervised clustering could result in the mixing of hot and cold data. To handle this problem, StreamCSD employs GC-guided remapping to reduce lifetime variance within each physical stream. GC-guided remapping reallocates outlier pages to dedicated streams based on their hotness.

**Identification of outlier pages in a stream.** Intra-SSD multi-stream management enables StreamCSD to detect page hotness by utilizing GC statistics. The typical GC process involves: 1) selecting a victim block; 2) migrating valid pages from the victim block to a new block; and 3) erasing the victim block. To identify outlier pages, StreamCSD tracks the migration count of each valid page. If a page undergoes excessive migrations, indicating its longevity and outlier status, it should be moved to a more appropriate stream. Empirical evaluation shows that 4 bits per migration count are sufficient, requiring only 128MB for a 1TB SSD.

**Remapping state transition.** StreamCSD designates two streams (*cold* and *frozen*) for outlier pages (refer to Figure 6). Pages are initially assigned to normal streams based on their stream IDs. If a page is migrated during GC, its *migration count* is incremented. When the count exceeds the *T1* threshold, the page is remapped to the cold stream. Cold stream pages are either returned to the original stream (with the *migration count* set to 0) or moved to the frozen stream. Pages in the frozen stream may move to the cold stream, setting their *migration count to T1*. The default *T1* and *T2* values are 1 and 2, based on empirical testing across various workloads.

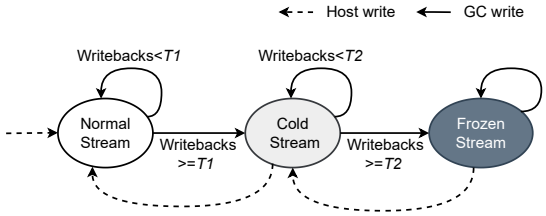


Fig. 6: The remapping state transition. *T1* and *T2* are configurable writeback thresholds.

### D. Secondary Lifetime-assisted Stream Mapping

In multi-application environments, the number of learned streams generated by the primary stream mapping may exceed the available physical SSD streams [39], [40]. We propose secondary lifetime-assisted stream mapping to tackle this issue. The main idea is to merge learned streams into the same physical stream based on their lifetime estimates.

***Learned stream* lifetime.** The learned stream's lifetime is estimated from its pages' lifetimes. However, tracking each page's lifetime requires significant memory, especially for high-capacity SSDs. To balance accuracy and memory consumption, we track lifetime at the chunk level. A chunk contains 256 consecutive pages, which share a timestamp table entry containing *chunk ID*, *learned stream ID*, and *timestamp*. If pages within a chunk belong to different learned streams, the chunk will have multiple entries, each corresponding to a different learned stream. Upon page updates or deletions, the timestamp table entry is adjusted, and new entries are created if none exist. Each update computes the chunk's lifetime by subtracting the *timestamp* from the current time, which is then used to update the learned stream's lifetime. By averaging the chunk lifetimes, we estimate the learned stream's lifetime.

**Stream mapping mechanism.** We designed a mapping mechanism to align learned streams with physical streams. Learned streams with similar lifetimes are grouped into the same physical stream, reducing intra-stream lifetime variance. A stream mapping table records the learned-to-physical mapping and other fields. Due to space limitations, we omit details.

## V. Evaluation

### A. Experimental Setup

We implemented StreamCSD on FEMU [20], a widely used NVMe SSD emulator for full-system research [35], [37]. StreamCSD is configured similarly to [35], with 32GB storage capacity, 8 channels, 8 planes per channel, 512 blocks per plane, 256 pages per block, and a 4KB page size. The Greedy policy [26], [38] is adopted as the default for GC victim selection. We also developed a real system prototype based on a PCIe 5.0 7.68TB CSD with a hardware compressor to further validate the functionality and efficiency of StreamCSD. We primarily use the FEMU emulator for comprehensive evaluation. The

TABLE I: Microbenchmark and representative real-world applications that are employed for evaluation.

| Benchmark | Description |
|---|---|
| Microbench (with public datasets) | Eight threads read data from real-world datasets and append to files with various data lifetimes. |
| RocksDB (DbBench) | LSM-tree based key-value store known for handling a large amount of sequential writes. |
| MySQL (TPC-C) | Relational database known for handling a large number of random writes. |
| SQLite (TPC-C) | Widely used embedded relational database with a lot of random writes. |

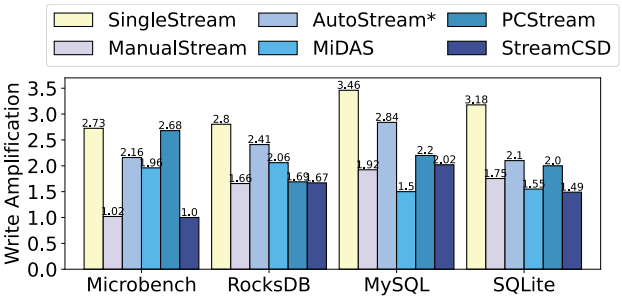


Fig. 7: Write amplification factor (WAF) (lower is better).

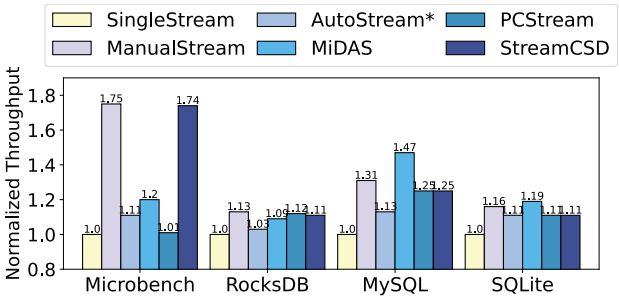


Fig. 8: SSD performance (higher is better).

hardware platform is used to evaluate AI workload performance and the actual overhead of learned stream management.

**Benchmarks.** We benchmark StreamCSD using a synthetic benchmark and three representative real-world applications listed in Table I. Microbench is designed with 8 threads, each sequentially writing datasets with varying entropy and lifetime. RocksDB performs random writes of 64 million key-value pairs, flushing WAL every 4 KB and compressing SSTables with the ZSTD algorithm. MySQL and SQLite are evaluated with TPC-C workloads, using 60 and 170 warehouses, respectively. The total data written exceeds the SSD capacity: 280GB for RocksDB and 120GB each for MySQL and SQLite, all hosted on the Ext4 file system.

**Counterparts.** We dedicated significant efforts to port state-of-the-art multi-streaming schemes listed in Table II. Particularly, AutoStream [39] leverages historical IO patterns for LBA-based stream identification. PCStream [19] identifies streams by using program context. MiDAS [23] segregates data by age and dynamically adjusts groups. To ensure fairness, write amplification and throughput metrics are evaluated under consistent resource conditions and configurations.

### B. WAF and Performance Evaluation

**WAF.** We begin the evaluation by comparing the Write Amplification Factor (WAF) improvements under Microbench, as shown in Figure 7. Significant WAF reductions are observed for ManualStream and StreamCSD, both achieving around 63.4% reduction compared to SingleStream. MiDAS and AutoStream* also show reductions of 28.2% and 20.9%, respectively. However, PCStream shows negligible improvement, as Microbench generates IO requests using multiple threads that share the same task handler, resulting in insufficient program context to distinguish datasets with different lifetimes.

For RocksDB, StreamCSD and ManualStream both demonstrate the highest WAF reductions of approximately 40%, compared to PCStream (39.6%), MiDAS (22.5%) and AutoStream* (13.9%). For MySQL, MiDAS achieves the highest WAF reduction of 56.64%, followed by ManualStream (44.5%), StreamCSD (41.6%), PCStream (36.4%), and AutoStream* (17.9%). MiDAS identifies hot data based on update intervals but struggles with out-of-place updates, as valid pages may be directly discarded. This limits MiDAS' efficiency in RocksDB, where out-of-place updates are more prevalent. However, it works effectively for MySQL, which has many in-place updates. In contrast, StreamCSD addresses this limitation by learning data content, enabling more accurate stream identification upon the first write. In SQLite, StreamCSD significantly outperforms ManualStream. Although ManualStream uses separate lifetime hints for rollback journals, WALs, and database files, it fails to distinguish between hot and cold data within a single database file. In contrast, StreamCSD employs GC-guided remapping, enabling more effective differentiation of intra-file hot and cold data.

**Performance.** There is a strong correlation between WAF and performance. A lower WAF means lower GC overhead. As illustrated in Figure 8, StreamCSD significantly outperforms SingleStream in various applications, delivering performance improvements of 74%, 11%, 25%, and 11% for Microbench, RocksDB, MySQL, and SQLite, respectively. Compared to MiDAS, StreamCSD provides higher throughput for Microbench and RocksDB with extensive out-of-place updates, whereas MiDAS performs better for MySQL and SQLite with numerous in-place updates.

### C. In-depth Analysis

**Effect of primary learning-based stream mapping.** We use RocksDB as a case study. As shown in Figure 7, we see that both StreamCSD and PCStream achieve near-optimal WAF reduction. To investigate further, we analyze the underlying mechanisms. RocksDB generates several types of files, including the Write-Ahead Log, level 0 SSTables (L0), and level 1 to n SSTables (L1–Ln). ManualStream assigns unique labels to these file types, allowing SSDs to manage them with dedicated Stream IDs for optimal performance. Similarly, PCStream identifies these streams by deriving program context from function call stacks, effectively differentiating WAL, L0, and L1–Ln. StreamCSD achieves a similar categorization by perceiving upper-level compression strategies: no compression for WAL, lightweight compression for L0, and standard compression for L1–Ln.

**Effect of GC-guided stream remapping.** We compare the WAF results with GC-guided remapping enabled and disabled across different workloads. Figure 9 shows that for MySQL and SQLite, where database files inherently mix hot and cold data, remapping reduces WAF by 22.9% and 21.2%, respectively. For RocksDB, remapping achieves a moderate WAF reduction of 5.6%, addressing SSTable level identification challenges. In contrast, for Microbench, remapping has minimal impact as data lifetimes within each learned stream have already converged. Futhermore, as shown in Figure 10, we analyze the remapping mechanism's sensitivity to threshold *T1*

TABLE II: Description of various multi-streaming schemes and a regular SSD as the baseline.

| Scheme | Description |
|---|---|
| SingleStream (Regular SSD) | Mix pages from all streams causing serious write amplification. (**Baseline**) |
| ManualStream [17] | Multi-stream, programmer manually tag stream identification. (**Programmer-dependent**) |
| AutoStream* | LBA-based multi-streaming. A ported FEMU implementation of AutoStream [39]. (**Historical IO pattern-dependent**) |
| MiDAS [23] (State-of-the-art) | Self-adjusting data grouping, segregating data blocks by age. (**Historical IO pattern-dependent**) |
| PCStream [19] (State-of-the-art) | Multi-stream, program context-based stream identification. (**Host-dependent**) |
| StreamCSD (**Our solution**) | Multi-stream, content learning-based stream identification. (**Host-transparent** ) |

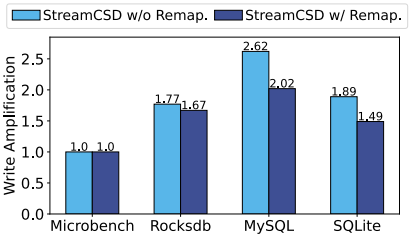


Fig. 9: Effect of GC-guided stream remapping and threshold sensitivity.

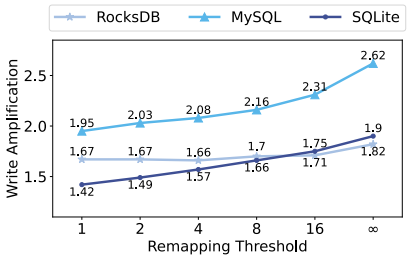


Fig. 10: Page copy-back ratio within physical streams.

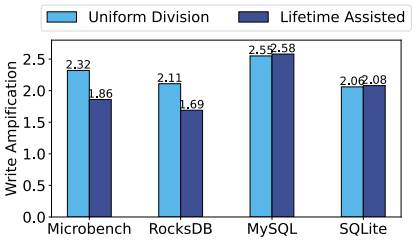


Fig. 11: Effect of secondary lifetime-assisted stream mapping.

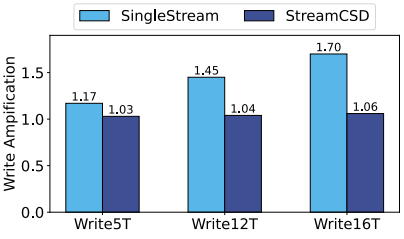


Fig. 12: WAF of a hardware StreamCSD with AI workloads.

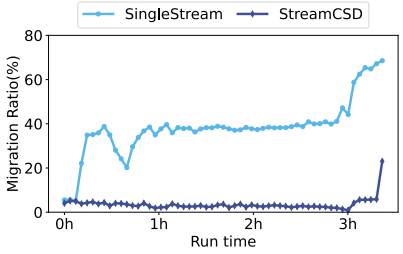


Fig. 13: GC migration ratio of a hardware StreamCSD with AI workloads.

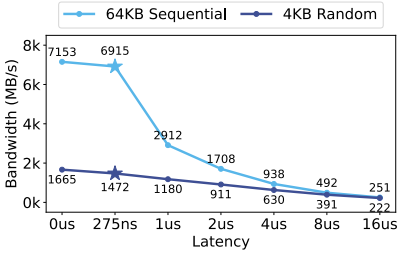


Fig. 14: Impact of stream management overhead on SSD write performance.

for RocksDB, MySQL and SQLite, respectively, with *T2* always set to *T1* + 1. Varying the threshold clearly influences WAF. Smaller thresholds lead to significant WAF reduction. At a *T1* threshold of 1, almost all workloads achieve the minimum WAF, which supports our default threshold settings.

**Effect of secondary lifetime-assisted stream mapping.** In a physical stream-limited scenario with 4 physical streams and 8 learned streams, we compare two merging strategies: one based on stream lifetime and the other on uniform division, where learned streams are evenly divided by IDs and mapped to physical streams. Figure 11 shows that, for Microbench and RocksDB, lifetime-assisted merging reduces WAF by 19.8% and 20.3%, respectively, highlighting the necessity of secondary mapping. However, this strategy does not improve other applications. For MySQL and SQLite, the relatively large page lifetime variance within learning streams limit the effectiveness of the lifetime-assisted strategy. It is worth noting that secondary mapping does not introduce significant side effects.

### D. Hardware StreamCSD for Generative AI Workloads

We notice that the compression ratio-based stream management is very effective in emerging generative AI multimodality applications. We evaluate the StreamCSD hardware prototype using data from *OpenAI* [3] and *Runway* [4]. We emulate multimodality applications generating and concurrently flushing *ChatGPT* txt, *DALL-E* png, *OpenAI TTS* mp3, and *Gen-2* mp4 data into a 7.68TB SSD. Without prior large-scale file lifetime knowledge, we assign the lifetime of these data streams in a 1:2:4:8 ratio. The SSD's built-in compressor is used to obtain the data compression ratio; therefore, the cost of feature extraction is negligible. As shown in Figure 12, the WAF of SingleStream increases with the growth in generative AI file writes, while StreamCSD's WAF remains almost unchanged. Figure 13 demonstrates the variation in GC migration ratio during the *Write-16T* test. GC process resulted in an additional 10.68 TB of data being written to SingleStream, with an average 38% migration ratio, leading to a WAF of 1.7. However, StreamCSD significantly reduces GC-written data. It categorizes the four types of AI files based on their compression ratios and directs them to distinct streams. With an average 4% migration ratio, StreamCSD notably reduces WAF compared to SingleStream.

### E. Impact of Learned Streaming on StreamCSD Performance

We evaluate the impact of learning-based stream management on write performance with the real PCIe 5.0 hardware SSD prototype. Learning-based stream mapping involves two components: a content-learning module for feature extraction and a clustering module for stream ID determination. We focus on the clustering module's impact, as the overhead of feature extraction is negligible due to the SSD's compression accelerator. Our customized streaming K-means algorithm takes *275ns* to assign a stream ID, resulting in a moderate performance impact—3.3% decrease for coarse-grain sequential writes and 11.6% for fine-grain random writes, as shown in Figure 14. These findings highlight the importance of lightweight feature extraction, efficient clustering algorithms, and hardware acceleration in intra-SSD stream management design.

## VI. Conclusion

Traditionally, reducing SSD write amplification and enhancing performance have depended on changes to interfaces or host-assisted stream ID labeling. However, in the age of AI and computational storage, it is crucial to advance SSD intelligence toward device-autonomous stream management for WA reduction. StreamCSD addresses this need by utilizing unsupervised content learning to eliminate dependency on the host. It autonomously assigns stream IDs to IO requests through content clustering. StreamCSD has been implemented and validated on both the FEMU emulator and a production-grade SSD hardware prototype equipped with a data compression accelerator, showcasing the feasibility and effectiveness of autonomous multi-stream SSDs. Although the current use of compression ratio as the feature representation poses limitations, preventing the complete elimination of the block interface tax, StreamCSD still significantly reduces this tax. Overall, StreamCSD exemplifies the transformative role of AI in storage technology, with significant implications for the future of SSD design.

## Acknowledgement

This work was supported by the National Key R&D Program of China (No.2023YFB4502901), the National Natural Science Foundation of China (under Grants 62372197, U22A2071, 62102155) and the Shenzhen Key R&D Program (No.KJZD20240903102459001). We sincerely thank anonymous reviewers and Prof. *Yu Hua* from HUST for their valuable comments.

## References


[1] "Awesome public datasets," https://github.com/awesomedata/awesome-public-datasets, accessed: 2023-11-15.

[2] "Compression — apache cassandra documentation," https://cassandra.apache.org/doc/stable/cassandra/operating/compression.html, (Accessed on 05/14/2024).

[3] "Text generation models," https://platform.openai.com/docs/guides/text-generation, accessed: 2023-12-20.

[4] "Video generation models," https://runwayml.com/, accessed: 2024-01-16.

[5] "Write ahead log (wal) · facebook/rocksdb wiki," https://github.com/facebook/rocksdb/wiki/Write-Ahead-Log-%28WAL%29, (Accessed on 05/14/2024).

[6] B. Abali, B. Blaner, J. Reilly, M. Klein, A. Mishra, C. B. Agricola, B. Sendir, A. Buyuktosunoglu, C. Jacobi, W. J. Starke, H. Myneni, and C. Wang, "Data compression accelerator on ibm power9 and z15 processors : Industrial product," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 1–14.

[7] H. Alwassel, D. Mahajan, B. Korbar, L. Torresani, B. Ghanem, and D. Tran, "Self-supervised learning by cross-modal audio-video clustering," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9758–9770, 2020.

[8] P. Bafna, D. Pramod, and A. Vaidya, "Document clustering: Tf-idf approach," in *Proceedings of the International Conference on Electrical, Electronics, and Optimization Techniques (ICEEOT)*, 2016.

[9] M. Bjørling, A. Aghayev, H. Holmberg, A. Ramesh, D. Le Moal, G. R. Ganger, and G. Amvrosiadis, "Zns: Avoiding the block interface tax for flash-based ssds," in *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, 2021, pp. 689–703.

[10] M. Bjørling, J. Gonzalez, and P. Bonnet, "Lightnvm: The linux open-channel ssd subsystem," in *15th USENIX Conference on File and Storage Technologies (FAST 17)*, 2017, pp. 359–374.

[11] W. Cao, Y. Liu, Z. Cheng, N. Zheng, W. Li, W. Wu, L. Ouyang, P. Wang, Y. Wang, R. Kuan *et al.*, "Polardb meets computational storage: Efficiently support analytical workloads in cloud-native relational database," in *Proceedings of the 18th USENIX Conference on File and Storage Technologies (FAST'20)*, 2020, pp. 29–41.

[12] J. Chang, L. Wang, G. Meng, S. Xiang, and C. Pan, "Deep adaptive image clustering," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 5879–5887.

[13] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with ycsb," in *Proceedings of the 1st ACM symposium on Cloud computing (SoCC'10)*, 2010, pp. 143–154.

[14] A. Grønlund, K. G. Larsen, A. Mathiasen, J. S. Nielsen, S. Schneider, and M. Song, "Fast exact k-means, k-medians and bregman divergence clustering in 1d," *arXiv preprint arXiv:1701.07204*, 2017.

[15] K. Han, H. Gwak, D. Shin, and J. Hwang, "Zns+: Advanced zoned namespace interface for supporting in-storage zone compaction," in *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, 2021, pp. 147–162.

[16] D. A. Huffman, "A method for the construction of minimum-redundancy codes," *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.

[17] J.-U. Kang, J. Hyun, H. Maeng, and S. Cho, "The multi-streamed solid-state drive," in *Proceedings of the 6th USENIX Conference on Hot Topics in Storage and File Systems (HotStorage'14)*, 2014.

[18] S. Karandikar, A. N. Udipi, J. Choi, J. Whangbo, J. Zhao, S. Kanev, E. Lim, J. Alakuijala, V. Madduri, Y. S. Shao, B. Nikolic, K. Asanovic, and P. Ranganathan, "Cdpu: Co-designing compression and decompression processing units for hyperscale systems," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–17.

[19] T. Kim, D. Hong, S. S. Hahn, M. Chun, S. Lee, J. Y. Hwang, J. Lee, and J. Kim, "Fully automatic stream management for multi-streamed ssds using program contexts." in *FAST*, 2019, pp. 295–308.

[20] H. Li, M. Hao, M. H. Tong, S. Sundararaman, M. Bjørling, and H. S. Gunawi, "The case of femu: Cheap, accurate, scalable and extensible flash emulator," in *16th USENIX Conference on File and Storage Technologies (FAST 18)*, 2018, pp. 83–90.

[21] Y. Lu, J. Shu, and W. Zheng, "Extending the lifetime of flash-based storage through reducing write amplification from file systems," in *11th USENIX Conference on File and Storage Technologies (FAST 13)*, 2013, pp. 257–270.

[22] C. Min, K. Kim, H. Cho, S.-W. Lee, and Y. I. Eom, "Sfs: Random write considered harmful in solid state drives," in *Proceedings of the 10th USENIX Conference on File and Storage Technologies*, ser. FAST'12. USA: USENIX Association, 2012, p. 12.

[23] S. Oh, J. Kim, S. Han, J. Kim, S. Lee, and S. H. Noh, "MIDAS: Minimizing write amplification in Log-Structured systems through adaptive group number and size configuration," in *22nd USENIX Conference on File and Storage Technologies (FAST 24)*, 2024.

[24] J. Ouyang, S. Lin, S. Jiang, Z. Hou, Y. Wang, and Y. Wang, "Sdf: Software-defined flash for web-scale internet storage systems," in *Proceedings of the 19th international conference on Architectural support for programming languages and operating systems*, 2014, pp. 471–484.

[25] E. Rho, K. Joshi, S.-U. Shin, N. J. Shetty, J. Hwang, S. Cho, D. D. Lee, and J. Jeong, "Fstream: Managing flash streams in the file system," in *Proceedings of the 16th USENIX Conference on File and Storage Technologies (FAST 18)*, 2018, pp. 257–264.

[26] M. Rosenblum and J. K. Ousterhout, "The design and implementation of a log-structured file system," *ACM Transactions on Computer Systems (TOCS)*, vol. 10, no. 1, pp. 26–52, 1992.

[27] C. Sabol and R. Stenfort, "Flexible data placement mode (fdp)," https://nvmexpress.org/wp-content/uploads/Hyperscale-Innovation-Flexible-Data-Placement-Mode-FDP.pdf, accessed: 2023-12-18.

[28] M. Satyanarayanan, "A study of file sizes and functional lifetimes," *ACM SIGOPS Operating Systems Review*, vol. 15, no. 5, pp. 96–108, 1981.

[29] ScaleFlux, "CSD 2000 Data Sheet," https://oss.scaleflux.com/202105/file_20210527_125844179.pdf, 2021, online; accessed 3 March 2022.

[30] D. Sculley, "Web-scale k-means clustering," in *Proceedings of the 19th international conference on World wide web*, 2010, pp. 1177–1178.

[31] Seagate, "Nytro 1000 SSD Series Datasheet)," https://www.seagate.com/www-content/datasheets/pdfs/nytro-1351-1551-sata-ssdDS1992-4-1907US-en_US.pdf, 2022, online; accessed on September 1, 2022.

[32] SNIA, "Computational storage architecture and programming model version 1.0," https://www.snia.org/sites/default/files/technical-work/computational/release/SNIA-Computational-Storage-Architecture-and-Programming-Model-1.0.pdf, 2022, online; accessed on September 3, 2022.

[33] M. Steinbach, G. Karypis, and V. Kumar, "A comparison of document clustering techniques," 2000.

[34] R. Stenfort, "Hyperscale storage perspective," https://storageconference.us/2023/StenfortPresentation.pdf, Santa Clara University, 2023.

[35] S. Wang, Z. Lin, S. Wu, H. Jiang, J. Zhang, and B. Mao, "Learnedftl: A learning-based page-level ftl for reducing double reads in flash-based ssds," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 616–629.

[36] Y. Wang, Z. Sun, Y. Zhou, T. Lu, C. Xie, and F. Wu, "Balloon-zns: Constructing high-capacity and low-cost zns ssds with built-in compression," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.

[37] Y. Wen, X. Zhao, Y. Zhou, T. Zhang, S. Yang, C. Xie, and F. Wu, "Eliminating storage management overhead of deduplication over ssd arrays through a hardware/software co-design," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2024, pp. 320–335.

[38] M. Wu and W. Zwaenepoel, "envy: a non-volatile, main memory storage system," *ACM SIGOPS Operating Systems Review*, vol. 28, no. 5, pp. 86–97, 1994.

[39] J. Yang, R. Pandurangan, C. Choi, and V. Balakrishnan, "AutoStream: Automatic stream management for multi-streamed SSDs," in *Proceedings of the 10th ACM International Systems and Storage Conference (SYSTOR'17)*, 2017.

[40] H. Yong, K. Jeong, J. Lee, and J.-S. Kim, "vStream: Virtual stream management for multi-streamed SSDs," in *10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 18)*, 2018.

[41] J. Ziv and A. Lempel, "A universal algorithm for sequential data compression," *IEEE Transactions on Information Theory*, vol. 23, no. 3, pp. 337–343, 1977.